

Manipal School of Information Sciences

Linux Operating System Theory Assignment

Linux Filesystem

Exercises 1-17: Storage, Hierarchy, Metadata and Investigation

Student Name:	Ribhav Devprasad
Reg. No.:	26110690002
Date:	3 November 2026

Linux Filesystem Investigation

How to use this document

> **Outputs must come from your own machine.** This exercise asks for your system's actual disks, partitions, mount points, usernames, timestamps and file sizes, and your own screenshots. Those cannot be known from outside, and inventing them would be fabricated evidence. Everything else is filled in: every command, every analysis answer, the purpose tables and the Exercise 17 scenario.
>
> **To finish:** (1) copy `filesystem_lab.sh` into your Ubuntu VM and run `chmod +x filesystem_lab.sh && ./filesystem_lab.sh`. It runs Exercises 1 to 16 in order, only reads system files, creates `~/cyber_lab` and saves `~/filesystem_lab_output.txt`. (2) Paste or screenshot each part of the transcript into the output boxes. (3) Fill the red italic cells from the transcript. Do not modify or delete any system files.

Exercise 1: Identify your Linux filesystem

```
$ lsblk
```

```
$ lsblk -f
```

```
$ df -T
```

```
$ df -h
```

Your output (run on your own machine and paste or screenshot here)

Observation table (one row per partition or mounted filesystem)

Device/Partition	Filesystem Type	Size	Mount Point
from lsblk -f / df -T			

Question	Your finding
1. Storage devices available	*e.g. the disk name shown with TYPE disk in lsblk*

2. Partitions on each device	*the part entries under each disk*
3. Filesystem type of each partition	*FSTYPE column of lsblk -f*
4. Device/partition holding /	*the row whose mount point is /*
5. Filesystem type of /	*Type column of df -T for /*
6. Total / used / available	*Size, Used, Avail of df -h for /*

Exercise 2: Disk, Partition, Filesystem, Mount Point

Draw your own machine using the values from Exercise 1 (do not copy the example values):

```
Disk:         (your disk, e.g. from lsblk)
  |
  +-- Partition:    (partition that holds /)
        |
        +-- Filesystem:   (type of that partition)
                |
                +-- Mount point:  /
```

Explanation (2 to 3 sentences, ready to use). A **disk** is the physical or virtual storage device. It is divided into one or more **partitions**, which are separate regions of the disk. A partition becomes usable when a **filesystem** (such as ext4) is created on it to organise files, directories and metadata. The filesystem is then attached to the single Linux directory tree at a **mount point** such as `/`, so that its files appear as part of the tree.

Exercise 3: Explore the root filesystem

```
$ pwd
```

```
$ cd /
```

```
$ pwd
```

```
$ ls
```

```
$ ls -d */
```

```
$ ls -A / | grep '^\.'
```

Your output (run on your own machine and paste or screenshot here)

Directories directly under / on a standard Ubuntu system: `bin boot dev etc home lib lib32 lib64 libx32 lost+found media mnt opt proc root run sbin snap srv sys tmp usr var` (some may differ by version; list yours from the `ls -d */` output). **Hidden entries:** `ls -A /` normally shows no entries beginning with a dot directly under `/`; note whatever your output shows. `ls -a` adds only `.` and `..`, which are special directory entries, not real hidden files.

Exercise 4: Filesystem hierarchy investigation

```
$
for d in /home /root /etc /var /var/log /tmp /usr /boot /dev /proc; do echo "== $d"; ls $d | head -n 8; done
```

Your output (run on your own machine and paste or screenshot here)

Standard example entries on Ubuntu are shown; confirm each against your own `ls` output and replace any that differ. `/root` normally gives *Permission denied* for a normal user, and that is itself a valid observation.

Directory	Example Contents	Purpose
`/home`	your username's directory (e.g. `ubuntu`)	Personal directories of normal users; holds their files and settings
`/root`	*Permission denied* as a normal user	Home directory of the superuser (root), kept separate from `/home`
`/etc`	`passwd`, `group`, `hosts`, `hostname`	System-wide configuration files
`/var`	`log`, `cache`, `lib`, `spool`, `tmp`	Variable data that changes while the system runs: logs, caches, queues, package databases
`/var/log`	`syslog`, `auth.log`, `kern.log`, `dpkg.log`	System and application log files
`/tmp`	session or application temp files and sockets	Temporary files, writable by all users and usually cleared at reboot
`/usr`	`bin`, `lib`, `share`, `local`	Installed programs, libraries and shared read-only data
`/boot`	`vmlinuz-*`, `initrd.img-*`, `grub`	Kernel images, initial RAM disk and bootloader files needed to start the system
`/dev`	`null`, `zero`, `tty`, `sda` (or similar)	Device files that represent hardware and pseudo-devices
`/proc`	`cpuinfo`, `meminfo`, numbered directories	Virtual filesystem exposing kernel and process information

Exercise 5: Find your home directory

```
$ whoami
```

```
$ echo $HOME
```

```
$ cd ~
```

```
$ pwd
```

```
$ ls
```

```
$ ls -a
```

Your output (run on your own machine and paste or screenshot here)

Item	**Your finding**
Username (`whoami`)	*from output*
Home directory (`echo $HOME`)	*from output*
Absolute path (`pwd`)	*from output*

`$HOME` is used so the location is not assumed; `cd ~` (or plain `cd`) moves to it, and `pwd` prints the absolute path.

Exercise 6: Hidden files investigation

```
$ ls
```

```
$ ls -a
```

```
$ ls -d .[!.]*
```

Your output (run on your own machine and paste or screenshot here)

Hidden entries commonly found in a home directory: `.bashrc`, `.profile`, `.bash_logout`, `.bash_history`, `.cache`, `.config`. List at least two that appear in *your* output.

1. **How does Linux identify a hidden file?** By its name: any file or directory whose name begins with a dot (`.`) is treated as hidden. There is no special hidden attribute.
2. **Why does a normal `ls` not show them?** `ls` skips names starting with a dot by default, to keep listings uncluttered by configuration files. `ls -a` (all) or `ls -A` shows them.
3. **What do `.` and `..` mean?** `.` is the current directory and `..` is its parent directory. Both exist in every directory.
4. **Can a hidden file still be accessed if its name is known?** Yes. Hiding only affects default listings; `cat .bashrc`, `cd .config` and similar commands work normally, subject to the usual permissions.
5. **Cybersecurity perspective:** A leading dot is only a naming convention, so it says nothing about safety. Legitimate hidden files hold user settings and history, while malware and attackers may also use dot-names to avoid casual notice (for example hidden scripts or backdoors in `/tmp` or startup files). Hidden files must therefore be inspected (owner, location, timestamps, content) before being judged safe or malicious.

Exercise 7: Build a small filesystem hierarchy

```
$ mkdir -p cyber_lab/{evidence/{network,system},logs,scripts,reports}
```

```
$ touch cyber_lab/evidence/network/traffic.txt cyber_lab/evidence/system/processes.txt
cyber_lab/logs/security.log cyber_lab/scripts/check.sh cyber_lab/reports/investigation.txt
```

```
$ tree cyber_lab        # if missing: sudo apt install tree   (or: find cyber_lab | sort)
```

Your output (run on your own machine and paste or screenshot here)

`mkdir -p` creates all missing parent directories, and brace expansion `{a,b}` lets one command create the whole tree. `touch` creates the empty files. Expected structure for verification:

```
cyber_lab
├── evidence
│   ├── network
│   │   └── traffic.txt
│   └── system
```

```
│       └── processes.txt
├── logs
│   └── security.log
├── reports
│   └── investigation.txt
└── scripts
    └── check.sh
```

Exercise 8: Absolute and relative paths

```
$ cd ~/cyber_lab && pwd
```

```
$ cd evidence/network && pwd      # relative
```

```
$ cd /etc && pwd                  # absolute
```

```
$ cd && pwd                       # shortest way home
```

```
$ cd cyber_lab/reports && pwd
```

```
$ cd .. && pwd                    # one level up
```

Your output (run on your own machine and paste or screenshot here)

1. **Absolute path:** a path that starts at the root directory `/` and therefore means the same thing wherever you currently are (e.g. `/etc/passwd`).
2. **Relative path:** a path interpreted from the current working directory, without a leading `/` (e.g. `evidence/network`, `../logs`).
3. **Difference between `/` and `~`:** `/` is the root of the whole filesystem tree. `~` is a shell shortcut for the current user's home directory (the value of `$HOME`).

Exercise 9: Identify file types

```
$ file /etc/passwd /etc /bin/bash /dev/null /proc ~/cyber_lab/logs/security.log ~/cyber_lab/scripts/check.sh
```

```
$ ls -ld /etc/passwd /etc /bin/bash /dev/null /proc ~/cyber_lab/logs/security.log ~/cyber_lab/scripts/check.sh
```

Your output (run on your own machine and paste or screenshot here)

The first character of `ls -l` gives the type: `-` regular file, `d` directory, `l` symbolic link, `c` character device, `b` block device.

Object	Type	Evidence (first character of ls -l / output of file)
`/etc/passwd`	Regular file (ASCII text)	`-`
`/etc`	Directory	`d`
`/bin/bash`	Regular file: ELF 64-bit executable (the shell program). On Ubuntu 20.04 `/bin` itself is a symlink to `usr/bin`	`-` (use `ls -ld /bin` to see the `l`)
`/dev/null`	Character special (device) file	`c`
`/proc`	Directory (mount point of the virtual `proc` filesystem)	`d`
`security.log`	Regular file, currently empty	`-`
`check.sh`	Regular file, currently empty (not executable until `chmod +x`)	`-`

Exercise 10: Examine file metadata

```
$ cd ~/cyber_lab/reports
```

```
$ echo "Investigation notes: suspicious login reviewed." >> investigation.txt
```

```
$ ls -l investigation.txt
```

```
$ stat investigation.txt
```

Your output (run on your own machine and paste or screenshot here)

Metadata item	Value from your stat output
Filename	*fill in*
File size	*fill in*
File type	*fill in*

Owner	*fill in*
Group	*fill in*
Inode number	*fill in*
Access time	*fill in*
Modification time	*fill in*
Change time	*fill in*

1. **What is metadata?** Data *about* a file, stored in its inode: size, type, owner, group, permissions, link count, timestamps and pointers to the data blocks. The filename is stored in the directory entry, not in the inode.
2. **Is metadata the same as file content?** No. Content is the data inside the file (here, the line of text). Metadata describes the file and can be read without reading the content.
3. **Which timestamp changes when the contents are modified?** The modification time (**mtime**) changes, and so does the change time (**ctime**), because the size changed. The access time (atime) is updated when the file is read.

Exercise 11: Explore /etc

```
$ ls -l /etc/passwd /etc/group /etc/hostname /etc/hosts
```

```
$ cat /etc/hostname
```

```
$ cat /etc/hosts
```

```
$ head -n 5 /etc/passwd
```

```
$ head -n 5 /etc/group
```

Your output (run on your own machine and paste or screenshot here)

1. **Information stored under `/etc`:** system-wide configuration: user and group databases, hostname and hosts mapping, network, service, package-manager, SSH and cron settings.
2. **/etc/passwd:** one line per account with seven colon-separated fields: username, password placeholder (`x`, the hash is in `/etc/shadow`), UID, GID, comment (full name), home directory and login shell.
3. **/etc/hostname:** the name of the machine, as a single line.
4. **Why is `/etc` security-sensitive?** Changing its files changes how the system behaves: accounts and privileges (`passwd`, `shadow`, `sudoers`), authentication and SSH settings, services started at boot,

name resolution (`hosts`). An attacker who can write there can gain persistence or redirect traffic, so access is tightly restricted and changes are monitored.

Exercise 12: Explore Linux log storage

```
$ ls -l /var/log
```

```
$ ls -ld /var/log/*/
```

```
$ ls -lt /var/log | head -n 8
```

```
$ for f in /var/log/*; do [ -r "$f" ] && echo "readable      $f" || echo "NOT readable  $f"; done
```

Your output (run on your own machine and paste or screenshot here)

Item	**Your finding**
Available log files	*e.g. syslog, auth.log, kern.log, dpkg.log*
Log directories	*e.g. apt, journal, installer (as listed)*
Recently modified	*top lines of ls -lt*
Readable as a normal user	*from the loop*
Not readable as a normal user	*from the loop* (commonly auth.log, syslog, btmp, journal)

1. **Why does Linux maintain logs?** To record system, service and security events (logins, errors, service start/stop, package changes) for troubleshooting, auditing and monitoring.
2. **Why are logs useful in incident investigation?** They provide a time-ordered record of what happened: failed and successful logins, privilege use with sudo, service crashes, new software, so an investigator can reconstruct the sequence of events and identify the source.
3. **Why might access to some logs be restricted?** Logs can contain sensitive data (usernames, IP addresses, commands, sometimes passwords typed by mistake) and attackers may want to read them to learn about the system or edit them to erase traces. Restricting access to root or the `adm` group protects confidentiality and integrity.

Exercise 13: Investigate /tmp

```
$ ls -la /tmp
```

```
$ ls -ld /tmp
```

```
$ ls -lat /tmp | head -n 8
```

Your output (run on your own machine and paste or screenshot here)

Question	Your finding
1. Files/directories exist?	*from ls -la /tmp*
2. Hidden files exist?	*entries beginning with . (e.g. .X11-unix, .ICE-unix are common)*
3. Owners of entries	*owner column*
4. Recently modified	*top of ls -lat*

The listing of `/tmp` itself normally shows `drwxrwxrwt`: writable by everyone, with the **sticky bit** (`t`) so users can delete only their own files.

Why can temporary directories matter in an investigation? `/tmp` is writable by every user and usually not cleaned until reboot, so attackers commonly drop downloaded tools, scripts, payloads and staged data there. Odd file names, hidden files, executable files and recent timestamps in `/tmp` are classic indicators of compromise. It is also cleared on reboot, so it should be examined early.

Exercise 14: Explore the /proc filesystem

```
$ head -n 12 /proc/cpuinfo
```

```
$ head -n 5 /proc/meminfo
```

```
$ echo $$
```

```
$ ls -d /proc/$$
```

```
$ mount | grep ' /proc '
```

Your output (run on your own machine and paste or screenshot here)

1. **/proc/cpuinfo:** details about each CPU core: model name, vendor, clock speed, cache size, flags and features.
2. **/proc/meminfo:** memory statistics: total, free and available RAM, buffers, cache, swap.
3. **Why do numbered directories appear in /proc?** Each running process has a directory named after its process ID (PID), containing details such as its command line (`cmdline`), open files (`fd/`), memory maps, status and environment. The directory for your shell is `/proc/<PID from echo $$>`.
4. **Is /proc an ordinary disk directory?** No. It is a virtual (pseudo) filesystem of type `proc` created by the kernel in memory; its contents are generated on demand, occupy no disk space and disappear at shutdown.
5. **Why is /proc useful in security investigation?** It shows live system state: running processes and their command lines, executables, open files and network connections, loaded modules. This helps find suspicious processes (for example one running from `/tmp` or with a deleted executable) without installing extra tools.

Exercise 15: Explore /dev

```
$ ls /dev | head -n 20
```

```
$ ls -l /dev/null /dev/zero
```

```
$ file /dev/null
```

Your output (run on your own machine and paste or screenshot here)

1. **What does `/dev` represent?** Device files: the interface to hardware (disks, terminals, USB) and to kernel pseudo-devices such as `null`, `zero` and `random`.
2. **Are the entries ordinary user files?** No. They are special files that talk to device drivers rather than store data on disk.
3. **File type of `/dev/null`:** a **character special file**: the first character of `ls -l` is `c`, and a major and minor number appear where the file size would be. It discards everything written to it and returns end-of-file on read. (`/dev/zero` is also a character device and returns endless zero bytes.)
4. **Why expose devices as files?** It follows the Unix principle that everything is a file: the same tools and system calls (`open`, `read`, `write`) and the same permission model work for devices and ordinary

files, which keeps programs simple and lets access be controlled with normal ownership and permissions.

Exercise 16: Filesystem space investigation

```
$ df -h /
```

```
$ df --output=source,size,used,avail,pcent /
```

```
$ du -sh ~
```

```
$ du -sh ~/cyber_lab
```

Your output (run on your own machine and paste or screenshot here)

Item	Your finding
1. Total filesystem capacity	*from output*
2. Used space	*from output*
3. Available space	*from output*
4. Utilisation percentage	*from output*
5. Space used by your home directory (`du -sh ~`)	*from output*
6. Space used by `cyber_lab` (`du -sh ~/cyber_lab`)	*from output*

Difference between `df` and `du`. `df` (disk free) asks the *filesystem* how much space it has in total, used and free, so it reports on a whole mounted filesystem. `du` (disk usage) walks a directory tree and adds up the space used by the files in it, so it reports on specific directories. The two can differ because `df` includes filesystem metadata, reserved blocks and deleted-but-still-open files, which `du` does not see. Block-size rounding also makes the size of a tiny file (such as the empty files in `cyber_lab`) show as 4 KB in `du` although it holds almost no data.

Exercise 17: Cybersecurity investigation scenario

A Linux machine is reported to show suspicious activity and only the filesystem may be examined first. All seven locations are worth investigating, in an order driven by volatility and value of evidence: volatile data first, then logs and configuration, then persistence in user areas. Examination must be read-only so evidence is not altered.

Location	Why investigate it	Security-relevant information expected
`/proc`	Live view of the kernel and running processes; lost at shutdown or reboot, so examined first	Running processes and their command lines (`cmdline`), executable path (`exe`), open files (`fd`), network connections (`net/tcp`), loaded kernel modules, environment variables
`/tmp`	World-writable and commonly cleared at reboot; a favourite staging area for attackers	Dropped scripts, downloaded tools, payloads, hidden dot-files, executables, odd names, very recent timestamps, unexpected owners
`/var/log`	Primary timeline of events	`auth.log`: failed and successful SSH logins, sudo use, new accounts; `syslog`/`kern.log`: service and kernel events; `dpkg.log`/`apt`: software installed; web or application logs; gaps or truncated logs that suggest tampering
`/etc`	Configuration shows what could have been changed to give access or persistence	`passwd`, `shadow`, `group`, `sudoers` (new or privileged accounts); `ssh/sshd_config`; `cron*` (scheduled tasks); `hosts` (redirects); services and startup scripts; modified timestamps
`/home`	User files and settings; the usual place for user-level persistence	Shell history (`.bash_history`), `.ssh/authorized_keys` (added keys), `.bashrc`/`.profile` (injected commands), hidden files and directories, downloads, user crontabs
`/root`	The superuser's home; evidence of privileged activity	Root's shell history, SSH keys, scripts and tools left by someone with root access (requires root to read)
`/dev`	Device files; ordinary files should not normally be here	Unexpected regular files or extra device nodes (a place to hide files, or a sign of a rootkit); mounted or removable devices

Submission checklist

Required item	Where
1. Commands used for each exercise	Shown under each exercise, and in `filesystem_lab.sh`
2. Relevant output or screenshots	Paste from `~/filesystem_lab_output.txt` or screenshot your terminal into the output boxes
3. Observation tables	Exercises 1, 4, 5, 9, 10, 12, 13, 16 (fill the red cells)
4. Disk to Mount Point diagram	Exercise 2 (use your own values)
5. Answers to analysis questions	Provided in each exercise